

Detect text duplication based on deep learning and hashing techniques

Group members :

1. Trần Trung Nghĩa
2. Hoàng Minh Hải
3. Phạm Minh Đức



Introduction: Text Deduplication

Large Language Models (LLMs) require vast amounts of high-quality data for training. However, raw training data often contains significant redundancy, posing a major challenge for AI companies building ambitious new language models.

Why Deduplication Matters

- **Wasted Resources:** Increases training time, costs, and energy.
- **Reduced Performance:** Models overfit, limiting generalization.
- **Memorization Effect:** Risk of verbatim reproduction, privacy leaks, copyright issues.
- **Misleading Evaluations:** Inflated metrics from duplicates in test sets.

Deduplication Approaches

- **Exact Matching:** Identifies identical duplicates via hashing.
- **Approximate Matching:** Finds near-duplicates using SimHash or MinHash LSH and Jaccard similarity.
- **Semantic Matching:** Identifies similar meaning using vector embeddings.

Traditional methods are computationally infeasible for terabytes of data. Innovative approximate methods like Hashing or Indexing balance recall and precision while managing costs.

Exact Matching: The Bloom Filter

A Bloom Filter is a space-efficient probabilistic data structure used to test set membership. It uses a bit array (initially all 0) and multiple hash functions.

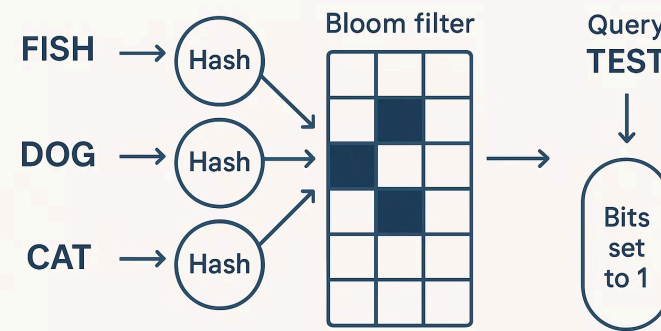
- **Insertion:** Hashes the item and sets corresponding bits to 1.
- **Query:** Checks if all bits for an item are 1. If it says "No," the item is definitely new (no false negatives).

It's highly memory efficient ($O(1)$ space) and fast ($O(k)$ constant time lookup).

Limitations & Mitigation

- False positives: Choose optimal filter size and hash functions.
- No deletion support: Use Counting Bloom Filters for dynamic removal

HOW A BLOOM FILTER WORKS



Approximate Matching: Hashing Strategies

1

SimHash (Hamming distance)

Uses "Random Hyperplanes" to generate a binary fingerprint.
Compares with Hamming Distance. Best for dense vectors
and short text.

၂၂

MinHash (Jaccard Similarity)

Measures overlap of n-grams (shingles) using random
permutations. Best for detecting containment and high
lexical overlap in documents.

SimHash: Compact Binary Fingerprints

SimHash is a Locality-Sensitive Hashing (LSH) technique that approximates cosine similarity using compact binary fingerprints. Unlike cryptographic hashes, SimHash maximizes collisions for similar inputs.

The Geometry of Random Hyperplanes

SimHash's core is Random Hyperplane Rounding. A hyperplane divides embedding space into two half-spaces. A hash function $h_r(v) = \text{sign}(v \cdot r)$ generates a bit (1 or 0) based on which side of a random hyperplane a vector v falls.

$$\Pr[h_r(u) = h_r(v)] = 1 - \frac{\theta}{\pi}$$

The probability of two vectors having the same hash bit relates directly to the angle θ between them. This allows converting expensive cosine similarity operations into fast bitwise **XOR operations (Hamming distance)**.

MinHash: Jaccard Similarity for Text

MinHash is an LSH scheme designed for Jaccard Similarity, a set-theoretic measure capturing the ratio of intersection to union of two sets. It's robust for detecting containment and overlap in text, independent of vector magnitude.

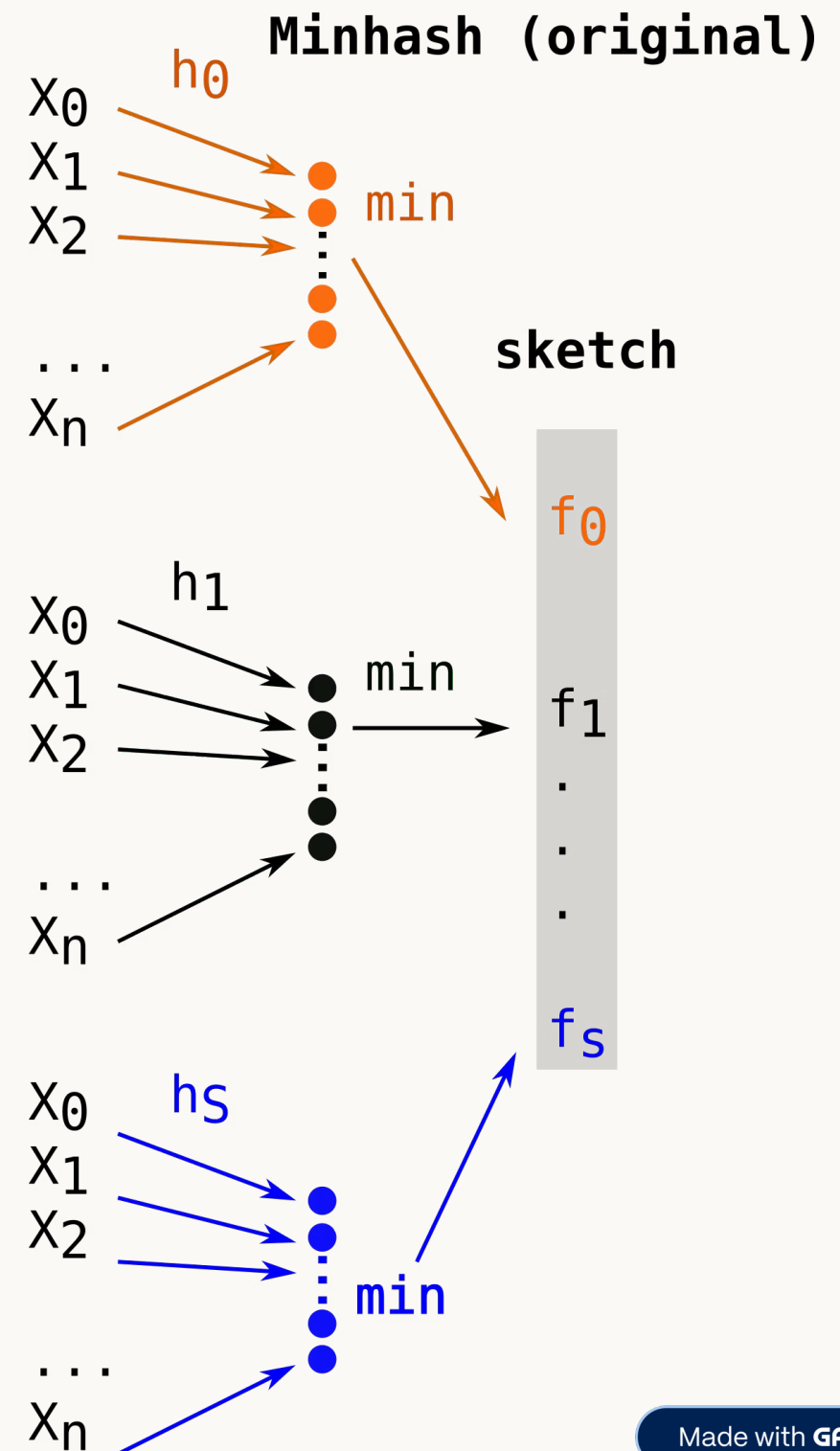
$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Min-Wise Independent Permutations

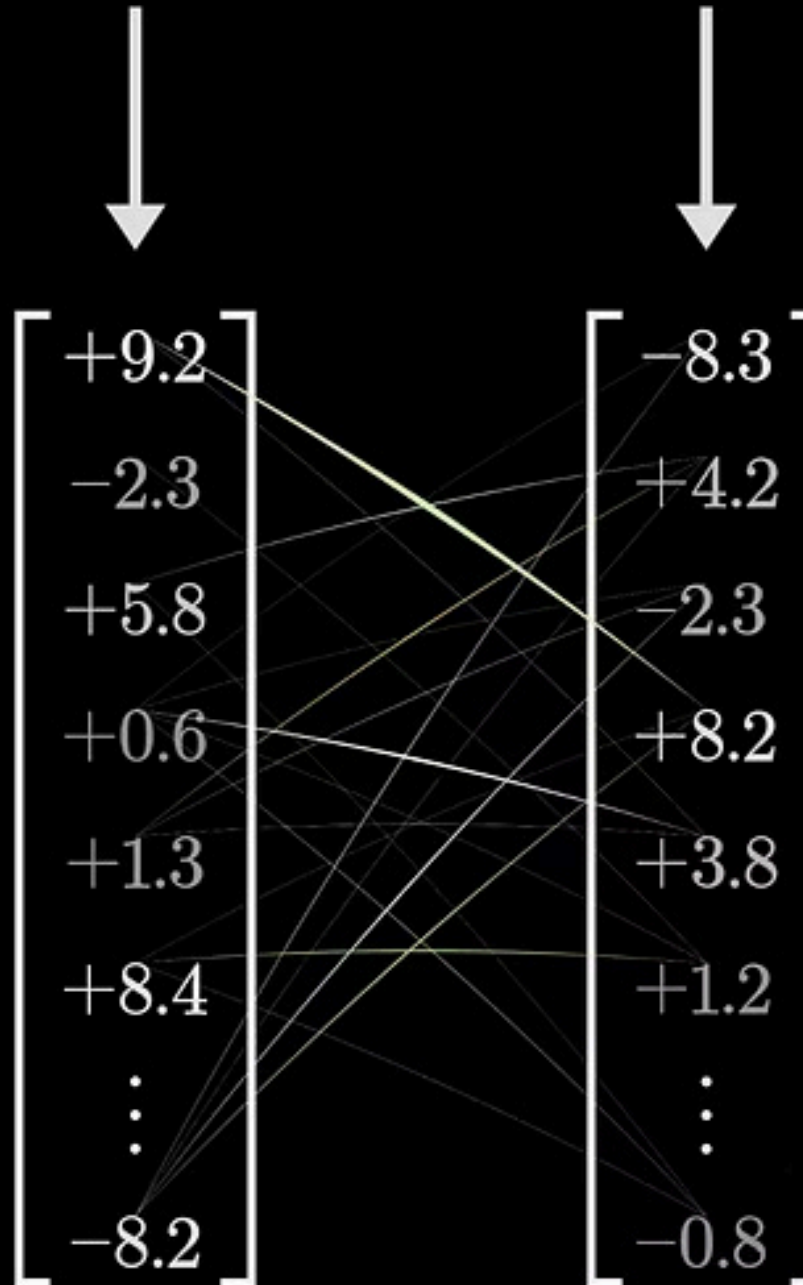
MinHash's theoretical basis lies in random permutations. For any two sets A and B, the probability that their MinHash values are identical equals their Jaccard similarity.

If $hmin(S)$ is the earliest element of set S in a random permutation, then $Pr[hmin(A) = hmin(B)] = J(A, B)$.

This is because $hmin(A) = hmin(B)$ if and only if the first element from $A \cup B$ in the permutation also lies in $A \cap B$. This probability is exactly the Jaccard similarity.



fluffy creat



Sentence-Transformers: Semantic Equivalence

Limitations of Bag of Words

- Fails to recognize semantic equivalence (e.g., "I love cars" vs. "I adore vehicles").
- Cannot handle synonyms, word order, or contextual meaning.
- Ignores word interactions in forming complete thoughts.

Modern NLP: Vector Embeddings

Modern NLP represents each sentence as a numerical vector (embedding), capturing semantic meaning in a high-dimensional space.

- Sentences with similar meanings are close.
- Sentences with different meanings are far apart.

This allows computers to measure semantic similarity using geometric concepts, converting duplicate detection into a vector comparison problem.

The Sentence-Transformers framework is used to produce high-quality embeddings for sentences, paragraphs, and long documents, a crucial step in vector generation.

FAISS: Facebook AI Similarity Search

FAISS (Facebook AI Similarity Search) provides an efficient solution for similarity search and nearest-neighbor retrieval in high-dimensional vector spaces. It's used in recommendation systems, image retrieval, clustering, and text deduplication.

How FAISS Works

01

Vector Index Construction

Sentence embeddings are collected into a specialized index for similarity search. FAISS supports various index types optimized for different dataset sizes.

02

Adding Vectors to the Index

Each embedding vector is added to the FAISS index. Clustering or quantization can be applied to structure the vector space efficiently.

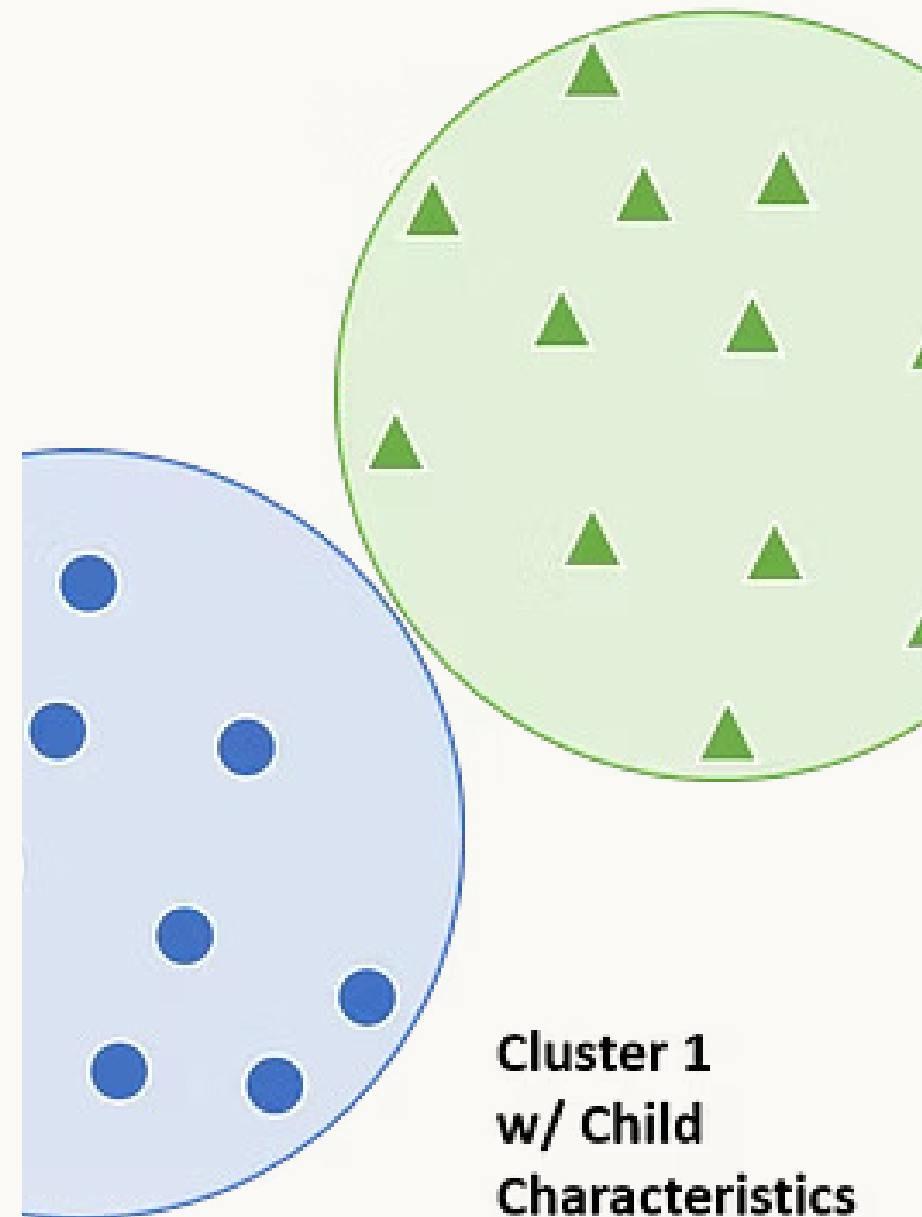
03

Querying

For a new query vector, FAISS searches a relevant subset of the index, computes approximate similarities, and returns the k most similar vectors. This Approximate Nearest Neighbor (ANN) approach balances high recall with low computation time.

FAISS efficiently indexes sentence embeddings (e.g., 384 or 768 dimensions) to rapidly identify similar existing vectors.

ustering



Height

Made with GAMMA

Dataset Overview: Quora Question Pairs (QQP)

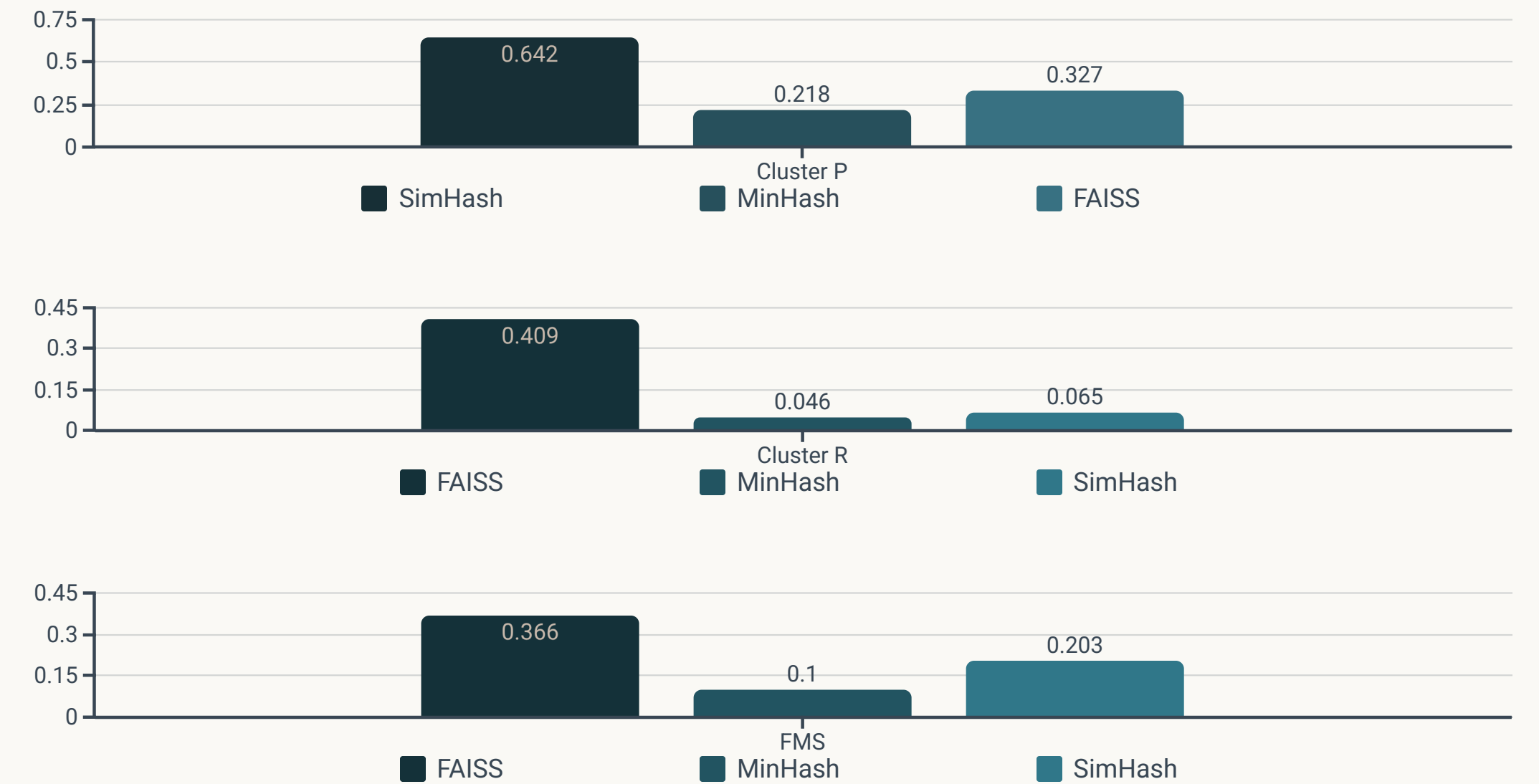
The Quora Question Pairs (QQP) dataset is a cornerstone benchmark in Natural Language Processing, famously included in the GLUE (General Language Understanding Evaluation) benchmark. It was released by Quora to solve a real-world product challenge: the "One Question" principle.

Dataset Statistics and properties

- **Volume:** The dataset contains approximately **404,290** annotated pairs of questions.
- **Class Imbalance:** It reflects real-world conditions where unique data is more common than duplicates:
 - ~37% are Positive (True Duplicates).
 - ~63% are Negative (Distinct Questions).
- **"Dirty" Data:** Because it comes from real users, the text is unstructured and noisy, containing typos, slang, inconsistent capitalization, and varying sentence structures.
- Unlike raw web-scraped data, QQP provides **verified labels** (`is_duplicate=1`).
- This allows us to mathematically calculate **Precision, Recall** to strictly evaluate our algorithms.
- It specifically challenges the "Lexical Gap" by containing non-duplicate pairs that share many words (e.g., "How does X affect Y?" vs "How does Y affect X?").

Performance Evaluation

Because deduplication is ultimately about grouping questions into equivalence classes, metrics defined on the induced clusters are considered. Cluster precision (Cluster P), cluster recall (Cluster R) and the Fowlkes–Mallows score (FMS) are computed on nodes whose cluster size is at least two



- ❏ **FAISS (Best Structure):** Highest FMS (0.37) → Best balance of purity and completeness. Successfully reconstructs long chains of duplicates (Transitivity).
- SimHash (High Precision):** Precision (0.64) → Clusters are very clean but small. However, fragmented, breaks large topics into tiny pieces (Low Recall).
- MinHash (Poor Fit): Lowest Scores** → Fails on short text, lexical overlap cannot capture the semantic paraphrases in QQP.

- **Github Respository Link:**

mhaih/
Hoc_phan_mo_rong_dsa_...

A project in DSA course in HCMUT

3
Contributors

0
Issues

0
Stars

0
Forks



 GitHub

GitHub – mhaih/Hoc_phan_mo_rong_dsa_deduptext: A project in DSA co...

A project in DSA course in HCMUT. Contribute to mhaih/Hoc_phan_mo_rong_dsa_deduptext development by creating an account o...



- **Google Collab Links:**

– FAISS-based semantic indexing:



 colab.research.google.com

Google Colab



– Hashing-based methods:



 colab.research.google.com

Google Colab

